GigSync

Technical Architecture Document

Introduction

GigSync is a modern, microservice-driven platform designed to bridge the gap between live music events and artist streaming analytics. By consolidating disparate data sources—ranging from ticket availability to artist popularity and song streaming distribution. GigSync provides a unified, highly scalable ecosystem for music fans and industry analysts alike.
The application is built on **Java 17** using **Spring Boot 4.0.6**, incorporating legacy protocol integration via **Apache CXF** (SOAP) and modern RESTful APIs via **Jakarta**. To orchestrate these services efficiently, the architecture relies on **Spring Cloud Gateway** as a single entry point and **Spring Eureka** for seamless load balancing.
Through a decoupled network of prosumers and providers, GigSync asynchronously processes complex queries. This ensures that users seeking event details, ticket comparisons, or streaming stats receive rapid, integrated insights through a single, well-documented gateway.

Goals of the Application

The primary objective of GigSync is to offer a comprehensive, real-time overview of the music event lifecycle and artist presence. Specifically, the application is designed to achieve the following goals:

- **Unified Ticket Aggregation & Price Comparison**:
  - The platform aims to eliminate the friction of comparing ticket prices by querying both official ticket providers (**Legacy Box Office**) and secondary markets (**Resellers**).
  - By executing these queries asynchronously, GigSync delivers a side-by-side comparison of ticket prices and seat options for any given global event ID.
- **Artist Performance Analytics**:
  - Beyond ticketing, the application provides deep insights into artists and their catalogs by fetching real-time view counts and trending tracks via the **Music Stats** database.
  - Users can quickly analyze an artist's reach and determine which songs are driving their current popularity.
- **Real-Time Streaming Availability Mapping**:
  - A key goal is mapping where an artist's music can be legally streamed.
  - The system cross-references the **Streaming Availability** database to display exactly which platforms (e.g., Spotify, Apple Music, Deezer) host specific songs.
- **High Performance and Scalability via Microservices**:
  - By breaking down functionalities into specialized microservices (such as the *Ticket Searcher* and *Artist Analyzer* prosumers), the application prevents single points of failure.
  - Asynchronous communication is leveraged during heavy operations—such as querying dual ticket databases or fetching multi-platform streaming statuses—to maximize system responsiveness and throughput.
- **Seamless, Documented API Access**:
  - GigSync aims to expose a clean, secure, and developer-friendly interface.
  - Through the central API Gateway, external developers and clients can access all downstream microservices via a consolidated Swagger UI hub.

Used Technologies

Spring Boot

Java version: 17

Spring Boot version: 4.0.6

Apache CXF

Java version: 17

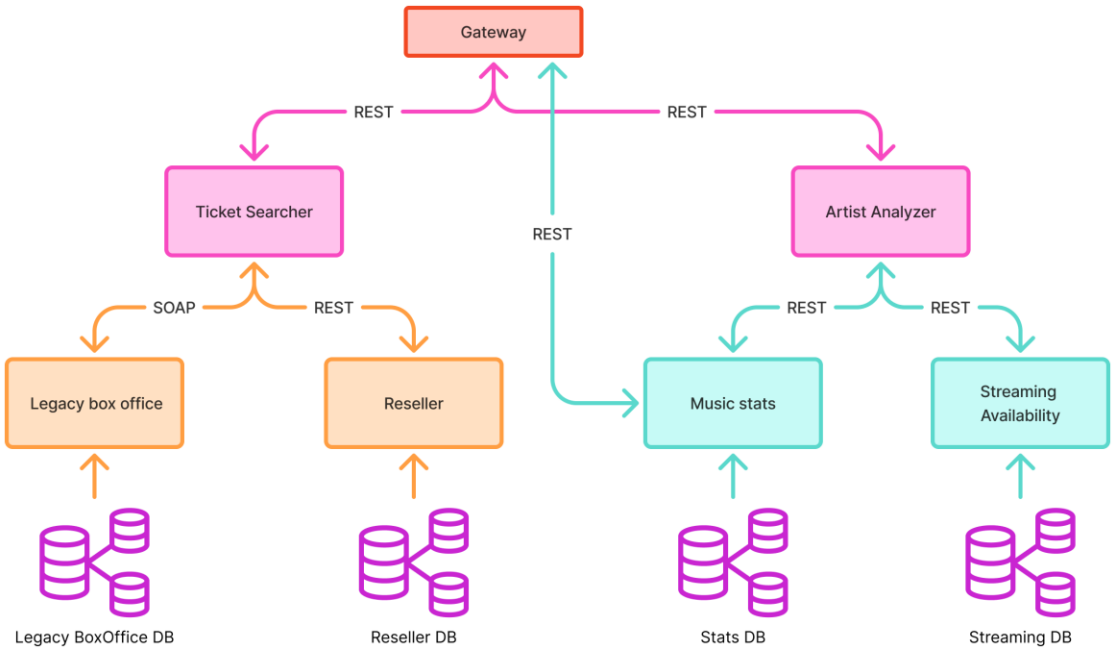
Jakarta

Java version: 17

Architectural Components

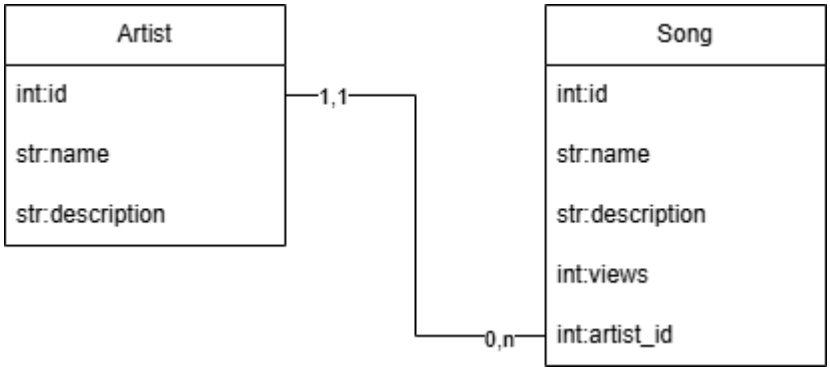
ID	Component	Role	Technology	Description
1	API Gateway	Gateway	Spring Cloud	Acts as the single entry point for all client-to-service interactions. Built with Spring Cloud Gateway.
2	Ticket Searcher	Prosumer	Spring (REST)	It queries both ticket providers (4, 5).
3	Artist Analyzer	Prosumer	Spring (REST)	Fetches the artist's trending tracks and overall information (6, 7).
4	Legacy box office	Provider	Apache CXF (SOAP)	A service exposing a SOAP interface returning official prices.
5	Reseller	Provider	Jakarta (REST)	A service returning secondary market prices.
6	Music stats	Provider/Prosumer	Spring (REST)	A service returning all info about the artist and his songs.
7	Streaming Availability	Provider	Spring (REST)	A service that returns if a song is available inside n streaming services.
8	Discovery service	Load Balancer	Spring Eureka	Load balancer for all microservices.

Architecture Diagram

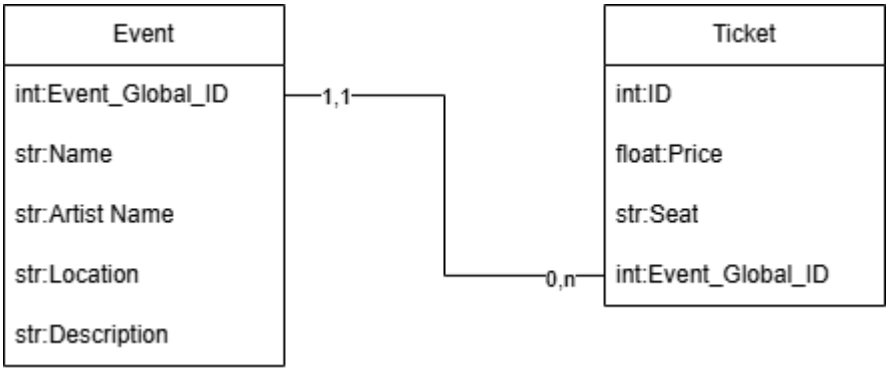


DB Diagram

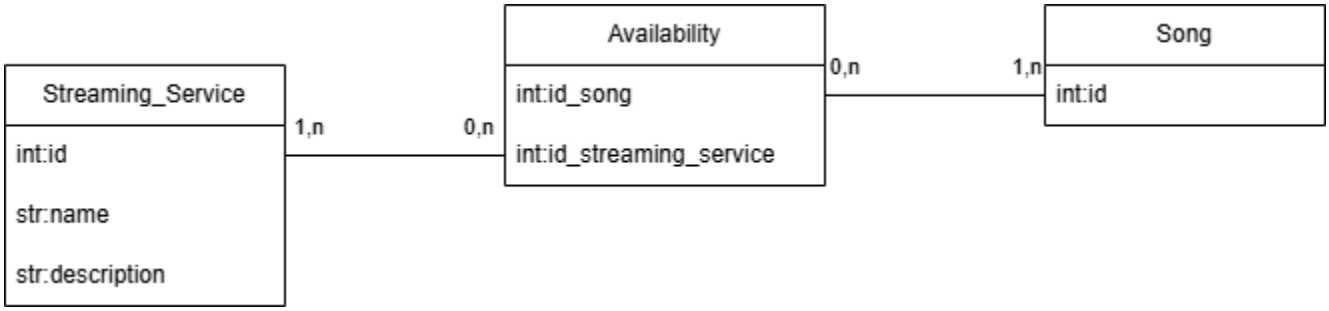
DB diagram for: Music stats



DB diagram for: Legacy box office, Reseller



DB diagram for: Streaming Availability



Legacy Box Office and Reseller DB

Event

Column Name	Data Type
Event_Global_ID	int
Name	str
Artist Name	str
Location	str
Description	str

Ticket

Column Name	Data Type
ID	int
Price	float
Seat	str
Event_Global_ID	Int

> *Ticket ID: unique for each event, so the same ID can't be in reseller and legacy box office.*
>
> *Event_Global_ID: unique — in different DBs the same event has the same ID.*
>
> *Relationship note: Event_Global_ID in the Ticket table has a many-to-one relationship (0,n to 1,1) with the Event_Global_ID column in the Event table.*

Music Stats DB

artist

Column Name	Data Type
id	int
name	str
description	str

song

Column Name	Data Type
id	int
name	str
description	str
views	int
artist id	int

> *Song ID: unique — in different DBs the same song has the same ID (e.g. Streaming and Music Stats DBs).*
>
> *Relationship note: artist id in the Song table has a many-to-one relationship (0,n to 1,1) with the id column in the Artist table.*

Streaming Availability DB

streaming_service

Column Name	Data Type
id	int
name	str
description	str

availability

Column Name	Data Type
id	int
id_song	int
is_streaming_service	int

song

Column Name	Data Type
id	int

Relationship notes: the Availability table serves as a junction table establishing a many-to-many relationship between Streaming_Service and Song.

id_streaming_service in Availability has a many-to-one relationship (0, n to 1, n) with id in Streaming_Service.

id_song in Availability has a many-to-one relationship (0, n to 1, n) with id in Song.

Endpoints

1: Gateway

Swagger UI: http://localhost:9000/swagger-ui/index.html

Endpoints

> *These are the endpoints visible by the user via the gateway.*

Involved Microservices	URL	Method	Description
2, 4, 5	/ticket-searcher/search	POST	Get all Tickets
2, 4, 5	/ticket-searcher/events/{Name}	GET	Searches by Name, or lists everyone if there is no Name
2, 4, 5	/ticket-searcher/events/{eventGlobalId}/tickets	GET	Compares official tickets (SOAP) and resold tickets (REST) for an event.
2, 4, 5	/ticket-searcher/events/{eventGlobalId}/tickets/cheapest	GET	Returns the cheapest ticket between officials and resold
3, 6, 7	/api/analyze/streaming-services	GET	Get all available streaming services
3, 6, 7	/api/analyze/songs	GET	Get all songs with associated streaming services
3, 6, 7	/api/analyze/songs/{id}	GET	Get a song by ID with its associated streaming services
3, 6, 7	/api/analyze/songs/by-name/{name}	GET	Get a song by Name with its associated streaming services
3, 6, 7	/api/analyze/songs/by-artist/{name}	GET	Get all artist's songs with their associated streaming services
3, 6	/api/analyze/artists	GET	Get all artists
3, 6	/api/analyze/artists/{id}	GET	Get artist by his ID
6	/api/stats/artists	GET	Get all artists
6	/api/stats/artists/{id}	GET	Get artist by his ID
6	/api/stats/songs	GET	Get all songs
6	/api/stats/songs/{id}	GET	Get a song by ID
6	/api/stats/songs/by-name/{name}	GET	Get a song by its Name
6	/api/stats/songs/by-artist/{name}	GET	Get all songs by artist Name
6	/api/stats/artists/by-name/{name}	GET	Get an artist by Name

2: Ticket Searcher (/ticket-searcher/)

ID	URL	Method	Description
1	/ticket-searcher/search	POST	Get all Tickets
2	/ticket-searcher/events/{Name}	GET	Searches by Name, or lists everyone if there is no Name
3	/ticket-searcher/events/{eventGlobalId}/tickets	GET	Compares official tickets (SOAP) and resold tickets (REST) for an event
4	/ticket-searcher/events/{eventGlobalId}/tickets/cheapest	GET	Returns the cheapest ticket between officials and resold

3: Artist Analyzer (/artist-analyzer/)

ID	URL	Method	Description
1	artists	GET	Get all artists
2	songs	GET	Get all songs
3	songs/{id}	GET	Get a song by its id
4	songs/by-name/{name}	GET	Get songs by name
5	songs/by-artist/{name}	GET	Get songs by their artist's name
6	streaming-services	GET	Get all available streaming services
7	artists/{id}	GET	Get artist by its id

4: Legacy box office

ID	URL	Method	Description
1	getEventByID/{Event_Global_ID}	GET	Get all info of an event by its ID
2	getAllEvents	GET	Get all available events
3	searchByName/{Name}	GET	Get all available events by the Name
4	getEventTickets/{Event_Global_ID}	GET	Get all available tickets by the Event ID
5	getTicket/{Event_Global_ID}/{Ticket_ID}	GET	Get Ticket info by his ID and the Event.

5: Reseller

ID	URL	Method	Description
1	getEventTickets/{Event_Global_ID}	GET	Get all available tickets by the Event ID
2	getTicket/{Event_Global_ID}/{Ticket_ID}	GET	Get Ticket info by the Ticket ID and the Event ID.

6: Music stats (/)

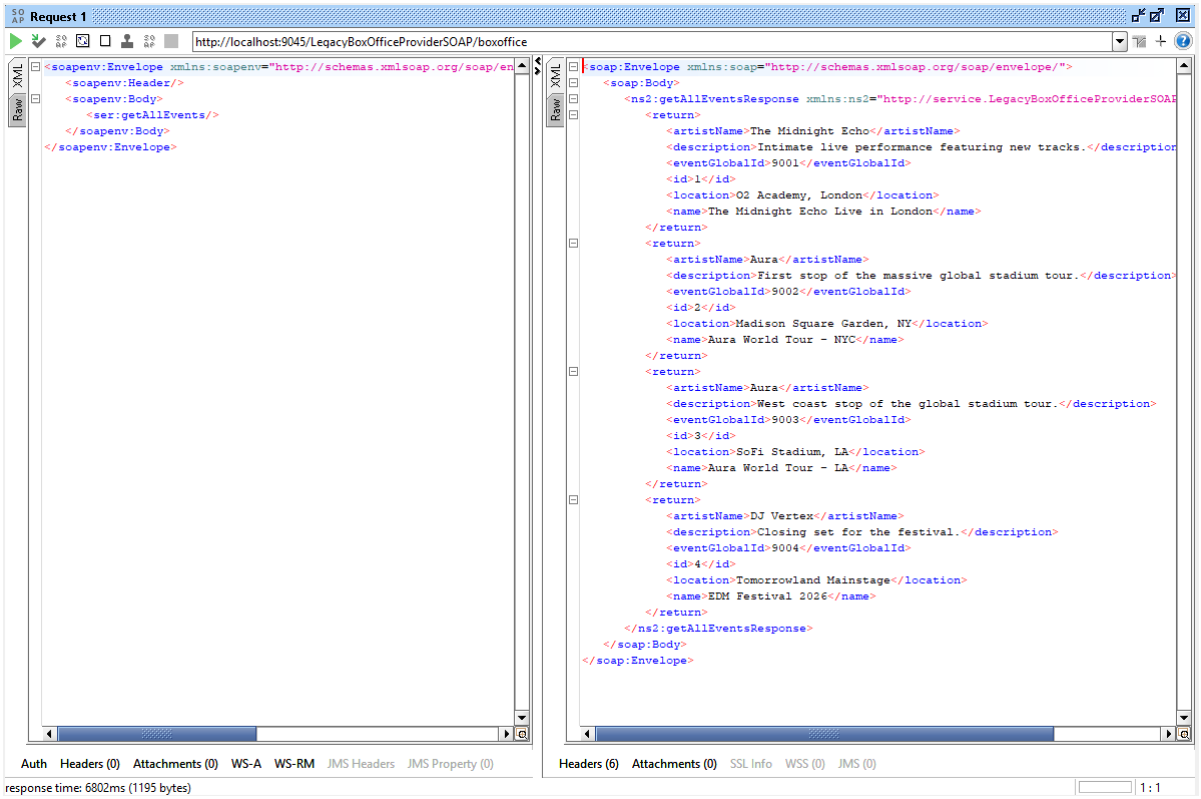
ID	URL	Method	Description
1	songs	GET	Get all songs
2	songs/by-name/{Song_Name}	GET	Search and get a Song by its Name
3	songs/by-artist/{Artist_Name}	GET	Search and get a Song by its Name
4	artists	GET	Get all artists
5	artists/{id}	GET	Get an artist by their id
6	artists/by-name/{Artist_Name}	GET	Search and get an Artist by his Name
7	songs/{id}	GET	Get a song by its id

7: Streaming Availability (/streaming-availability/)

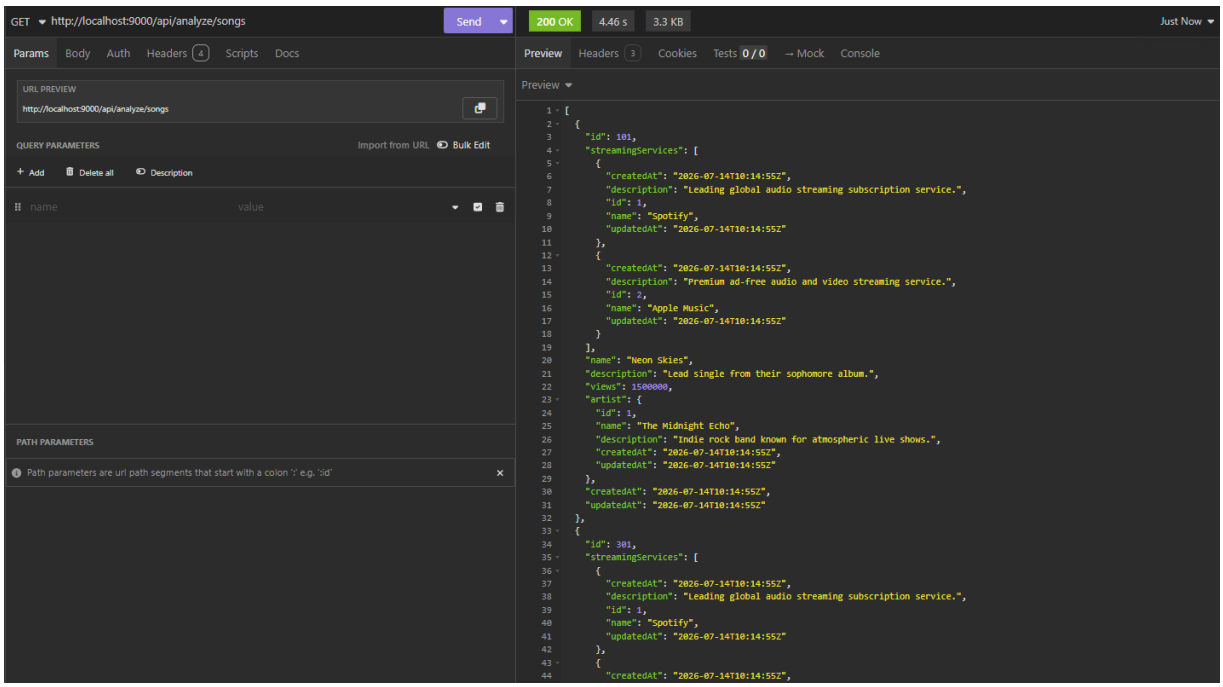
ID	URL	Method		Description
1	get-song-availability/{Song_ID}	GET		Gets streaming availability for all songs of the requested artist
2	get-all-songs-for-service/{Service_ID}	GET		Get all songs available for a specific streaming service
3	get-all-streaming-services	GET		Gets all streaming services available in the system

Examples

SOAP request: getAllEvents



REST request: getAllSongs



Frontend

- Is realized with NextJs and React
- Is available at this address: http://localhost:3000/

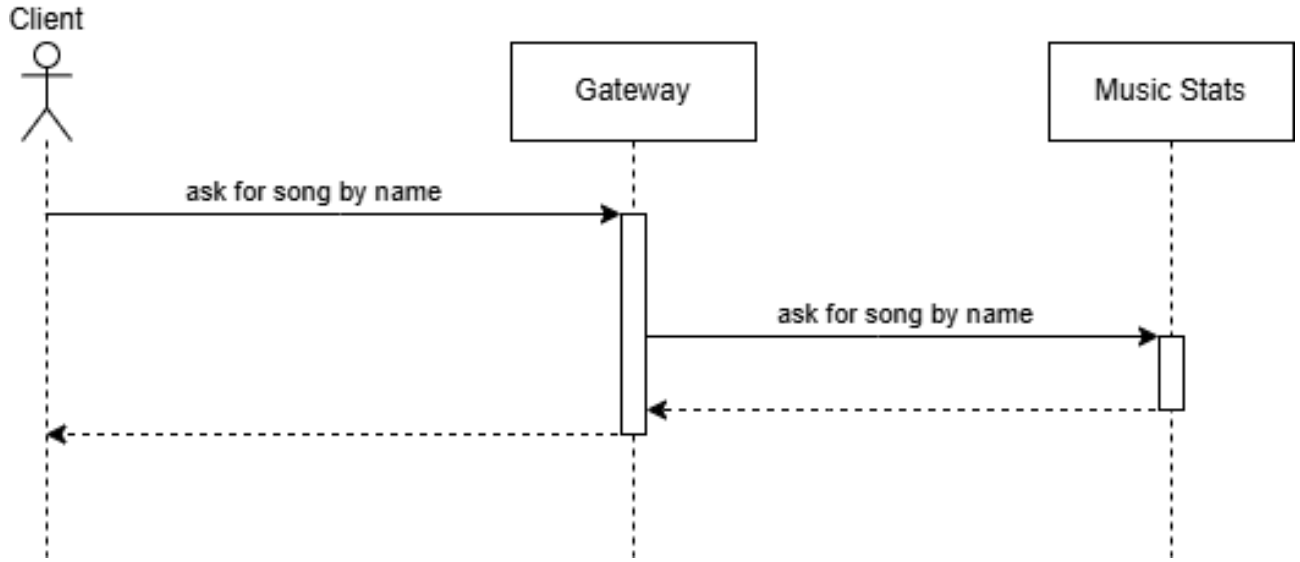
Asynchronous Communication

The asynchronous communication is implemented inside:

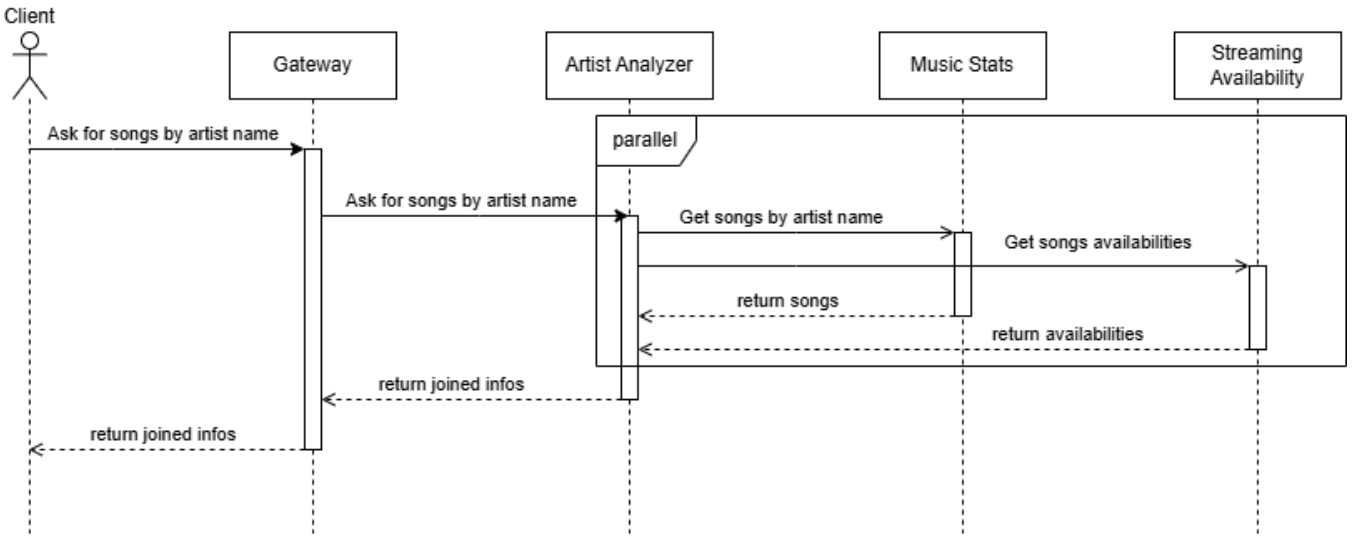
- **Ticket Provider** — Calls two different services (4, 5) to get the price of the tickets and get the available events.
-

Artist Analyzer — Calls two different services (6, 7) to get the artist's songs and their streaming availability.

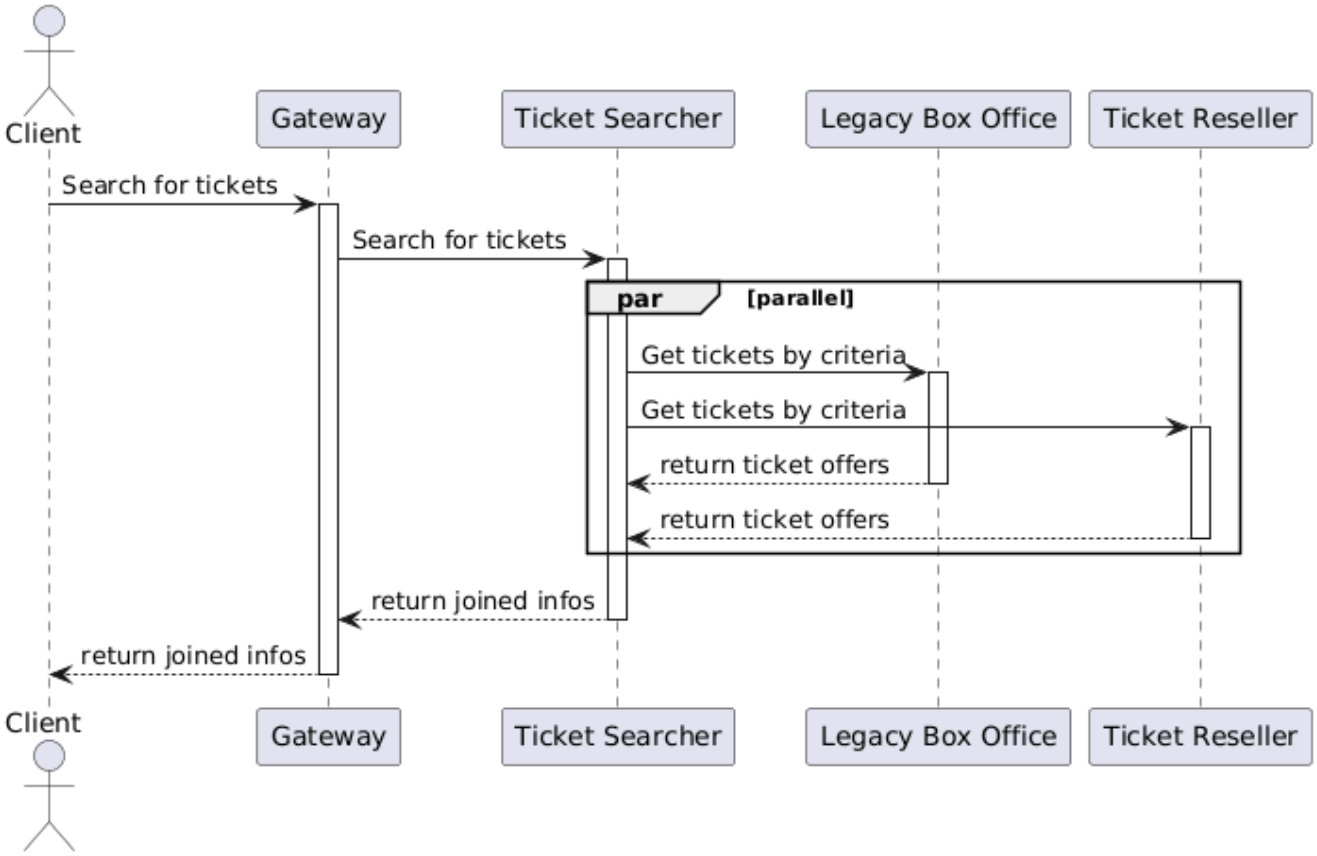
Interacting Scenarios



This sequence diagram illustrates the core flow of the Music Stats. It performs the single operation of getting the songs by the artist's name and returns a list of songs.



This sequence diagram illustrates the core flow of the Artist Analyzer. It requests songs and wraps the responses from the Music Stats (specifically the songs by artist name) and the Streaming Availability (specifically the songs availabilities) providers, joining them and returning the joined info.



This sequence diagram illustrates the core flow of the Ticket Searcher. It wraps all request types, either searching by criteria or searching by a specific ID, while the basics are kept the same: the prosumer orchestrates parallel requests to the Legacy Box Office and Ticket Reseller providers, merges the data from both sources and unifies the response. In the end, that information might also be passed through a filter to determine the cheapest ticket from them all.

Running the project

1. Clone the project

It's available on github, choose a suitable folder (preferably a path without spaces) and execute:

```
git clone https://github.com/Giacomix02/Service-oriented-Software-Engineering.git
```

2. Change directory to ./Service-oriented-Software-Engineering/Exam

3. Execute the .bat file “Build everything and launch docker compose.bat”

If you’re not on Windows (or something doesn't work), you can just execute for each folder in the project (except for DB, courseSlides, doc, frontend):

```
.\mvnw package -DskipTests
```

And then (in the *Exam* root folder):

```
Docker compose up --build
```

4. Wait a long time

5. Access the frontend by browser on `localhost:3000`

Feel free to explore the application and its features.
Microservices availability info is accessible through the discovery service on `localhost:8761`.
Swagger documentation for each exposed endpoint from the Gateway is available on `http://localhost:9000/swagger-ui/index.html`.